PYTHON FOR SRE / DEVOPS

30-Day Hands-On Mentorship Roadmap

Mission
Build strong Python fundamentals and practical independence for Site Reliability Engineering and DevOps work: understand code, write scripts from scratch, automate Linux and cloud tasks, debug failures, and explain engineering decisions.

Duration 30 days	**Daily commitment** 2 hours	**Total practice** ~60 hours
Learning style Concepts + coding + scenarios	**Primary focus** SRE / DevOps automation	**Final output** Capstone toolkit

A realistic promise

Thirty days can establish a strong foundation and meaningful hands-on independence. Advanced proficiency develops through continued project work, production exposure, and repeated practice beyond this plan.

Mentorship principle: learn the concept, solve a problem without copying, test your code, debug it, then explain why it works.

1. Learning outcomes

• Read and understand Python code written by other engineers.

• Write scripts from scratch using variables, collections, conditions, loops, and functions.

• Build reusable programs with modules, classes, exceptions, and file handling.

• Automate Linux operations and inspect processes, filesystems, logs, and network services.

• Work with REST APIs, JSON/YAML, AWS APIs, and Kubernetes concepts.

• Add logging, tests, configuration, and sensible error handling to automation tools.

• Explain design choices and troubleshoot your own code.

2. Daily two-hour structure

Time	Activity	What good looks like
20 min	Concepts	Explain the idea in your own words.
25 min	Guided examples	Trace code and predict its output.
40 min	Independent coding	Build without copying the example.
20 min	SRE scenario	Apply the concept to an operational problem.
15 min	Review and interview practice	Debug, summarize, and answer questions.

3. The 30-day curriculum

Week 1 — Python fundamentals (Days 1–7)

Day	Topic	Hands-on assignment
1	Setup, variables, data types, print()	Server information script
2	Operators and type conversion	Resource usage calculator
3	Strings and string methods	Log message analyser
4	Lists and tuples	Server inventory manager
5	Sets and dictionaries	Infrastructure inventory
6	if / elif / else and conditions	Disk threshold alert
7	for / while loops and nested loops	Multi-server health checker

Week 2 — Intermediate Python (Days 8–14)

Day	Topic	Hands-on assignment
8	Functions and parameters	Reusable health-check functions
9	Arguments, return values, scope	Modular monitoring script
10	List/dict comprehensions	Log filtering
11	Modules and packages	Reusable utility package

Day	Topic	Hands-on assignment
12	File handling and pathlib basics	Read and analyse Linux logs
13	Exceptions and error handling	Robust automation script
14	OOP: classes, objects, inheritance	Server monitoring classes

Week 3 — Python for Linux and SRE (Days 15–21)

Day	Topic	Hands-on assignment
15	Iterators and generators	Process large log files
16	Decorators and context managers	Execution timer and safe resource handling
17	os, pathlib, filesystem operations	Filesystem automation
18	subprocess and command execution	Run Linux commands from Python safely
19	psutil and system metrics	CPU, RAM, disk and process monitor
20	Regular expressions	Extract and summarize production log errors
21	Networking and sockets	Port availability checker

Week 4 — Advanced Python and DevOps (Days 22–30)

Day	Topic	Hands-on assignment
22	Threads and multiprocessing	Concurrent health checks
23	REST APIs and HTTP requests	API health checker
24	JSON, YAML and configuration	Infrastructure configuration parser
25	AWS Boto3 fundamentals	EC2 inventory automation
26	Kubernetes API fundamentals	Pod and deployment inventory
27	Logging, pytest and code quality	Tested monitoring utility
28	Project integration	Linux health monitoring tool
29	Cloud automation project	AWS resource monitoring tool
30	Capstone, review and assessment	SRE automation toolkit

4. Portfolio projects

Project 1 — Linux Server Health Monitor

• CPU, memory, disk and inode usage

• Process checks and configurable thresholds

• Alerts and timestamped log output

• Clear exit codes and error handling

Project 2 — Production Log Analyzer

• Read large log files safely

• Filter ERROR and WARNING entries

• Count recurring errors and summarize patterns

• Generate a readable report

Project 3 — AWS Resource Automation

• List EC2 instances with Boto3

• Identify stopped instances and report metadata

• Export an inventory report

• Use least-privilege IAM and avoid unintended changes

Project 4 — Kubernetes Health Checker

• Retrieve pod and deployment information

• Identify unhealthy or failed pods

• Summarize readiness and desired/current replicas

• Use read-only access for the initial version

Project 5 — SRE Automation Toolkit

• Combine monitoring, logging, API checks, and configuration

• Separate reusable modules from CLI entry points

• Add structured logs, tests, and documentation

• Provide safe defaults and actionable error messages

5. Development environment

• Python 3 (use a currently supported release)

• VS Code or another editor

• Linux terminal; Ubuntu/WSL or a Linux lab is suitable

• Git and a GitHub repository

• A Python virtual environment for each project

• AWS account only when needed; check costs and permissions before creating resources

• Libraries introduced when appropriate: requests, psutil, boto3, PyYAML, pytest

Start with the Python standard library. Learn the underlying concepts before relying on third-party packages.

6. Assessment and progression gates

Area	Expected standard by the end of the programme
Fundamentals	Explain and use core data types, control flow, collections, and functions without copying.
Problem solving	Break a small requirement into steps, implement it, and test edge cases.
Code structure	Use modules, functions, clear names, and appropriate classes.
Debugging	Read tracebacks, isolate the failing line, form a hypothesis, and verify the fix.
SRE automation	Build a safe Linux monitor and explain thresholds, exit codes, and logs.
Cloud/API work	Use documentation to build read-only AWS/API inventory scripts.
Capstone	Deliver a working, documented, tested toolkit and explain its design.

Weekly checkpoint: 10 concept questions, 5 coding problems, 2 SRE scenarios, and 1 practical assignment. Review weak areas before moving on.

7. Day 1 — first coding challenge

Topic: Python setup, variables, data types, print(), type(), and type conversion.

Write a script that displays the following server details using variables:

```
Server Name: prod-web-01
Environment: Production
CPU Cores: 8
Memory: 16 GB
Disk Usage: 85.5
Server Running: True
```

Then complete these tasks:

• Print each variable and its data type.

• Calculate total memory across 3 identical servers.

• Calculate remaining disk space if the disk capacity is 100 GB.

• Use meaningful variable names and add brief comments where helpful.

• Run the script, inspect the output, and explain each line in your own words.

8. Mentorship rules

• Attempt exercises before asking for a full solution.

• Share your code and error messages for review; remove secrets and credentials first.

• Expect line-by-line feedback on correctness, readability, safety, and maintainability.

• Do not move forward just because a topic was read; demonstrate it with code.

• Use official documentation when working with AWS, Kubernetes, and external libraries.

• Prefer read-only automation first. Never run destructive commands or cloud changes without reviewing the impact.

Final target

By Day 30, aim to independently understand a small SRE automation requirement, design a solution, write and test the Python code, debug it, and explain why it works. This is a foundation milestone—not a claim of mastery. Continue building projects after the 30 days.